Compliant Semantic Failures: Testing Agent-Facing Compatibility of Evolving Tool APIs

Anonymous Author(s)

***Abstract*—APIs are typically treated as compatible when schemas, generated clients, and regression tests remain stable. We show that tool-using large language model (LLM) agents expose a different compatibility boundary: an API can remain callable under the same schema while silently changing the business semantics that an agent relies on. We call this *agent-facing compatibility* and study it with an exposure-aware paired protocol over TAU-BENCH retail and airline. Our controlled mutations reveal *compliant semantic failures*—valid calls to unchanged schemas whose final task state violates an evolved rule—with 44 of 130 strict schema/client-compatible paired records becoming agent-breaking. Across 1,815 paired cells, semantic visibility is the decisive recovery channel: success rises from 50.1% under silent drift to 84.7% under any visible condition, while matched unused-tool controls show that the same drift is far less harmful off the agent's execution path. We ground the taxonomy in 151 public API-evolution entries across nine providers and replay three public-changelog-derived before/after cases, reproducing the same silent-versus-visible mechanism. A 288-cell control further shows that extra reasoning and reflection rarely recover from silent drift, whereas making the evolved rule visible restores success. These results call for exposure-aware semantic testing and CI/CD compatibility gates that check the business rules agents actually exercise, not only the schemas they call.**

***Index Terms*—API compatibility, LLM agents, tool APIs, regression testing, mutation testing, software evolution**

I. Introduction

API evolution is a normal part of software engineering. Service teams add fields, change defaults, revise eligibility policies, deprecate parameters, and harden operational constraints. For conventional clients, compatibility is commonly mediated by interface-description standards, schema validation, versioning conventions, API-evolution practices, and regression tests [1]–[5]. Those mechanisms give API and client teams a shared language for deployment risk.

Tool-using LLM agents disturb this contract. An agent is not only a typed caller of a function signature; it is also a natural-language client that reads tool descriptions, infers business rules, selects actions under task intent, reacts to runtime feedback, and carries hidden assumptions from prompts and previous observations. An API evolution can therefore preserve the JSON schema and typed-client call surface while still changing the behavior that the agent must infer to complete the task. Conversely, some changes that look breaking to a strict static checker can be recoverable if the agent receives a visible, actionable error at runtime.

This gap creates an industrial coordination problem. An API team may claim that a tool upgrade is backward compatible because generated clients still compile and existing calls remain valid. An agent platform team may still observe task regressions because the agent keeps following an obsolete business rule. This failure mode does not require an engineer to forget updating a prompt or an agent to issue an invalid tool call. The API release may satisfy the compatibility checks available to the deployment pipeline, yet those checks encode the wrong client model for tool-using agents. Compliant semantic failure is therefore a system-level compatibility defect: the release is safe under schema-level criteria but unsafe under the task-level semantics actually exercised by the agent. We argue that the missing concept is *agent-facing compatibility*: whether the evolved API preserves task-level success for LLM agents on tasks they could solve before the evolution.

The key visibility boundary is not whether a service provider documents a change somewhere, but whether the deployed agent observes it at task time. A changelog, migration guide, dashboard warning, or developer email can be visible to human maintainers while remaining absent from the agent's tool schema, tool description, and current API response. If the changed business rule is not propagated into those runtime channels, the agent can continue to issue a schema-valid call under an obsolete semantic assumption. Fig. 1 illustrates this visibility boundary. The left side shows developer-visible release communication; the right side shows the narrower runtime surface available to a deployed agent. For unit, default, computation, or delayed-enforcement changes, the local call may have no error channel: it can succeed while computing a different state or violating a downstream business invariant. Section IV later grounds this mechanism in public API-evolution records and Stripe-derived before/after replays using deterministic local wrappers.

This paper studies agent-facing compatibility as a software-engineering compatibility problem rather than a generic LLM benchmark problem. Traditional schema-level, typed-client, and behavioral compatibility remain necessary, but they are insufficient for agent clients because agent behavior also depends on execution exposure and semantic observability. Existing agent benchmarks largely evaluate success under fixed tool APIs; our study asks what happens when the tool API evolves while preserving the surface that the agent sees.

We make three contributions.

1) **Agent-facing compatibility.** We define a task-level compatibility layer for evolving tool APIs and identify compliant semantic failures, where a syntactically valid call to an unchanged schema violates evolved business semantics.
2) **An exposure-aware paired protocol grounded in API evolution.** We organize API changes into sur-

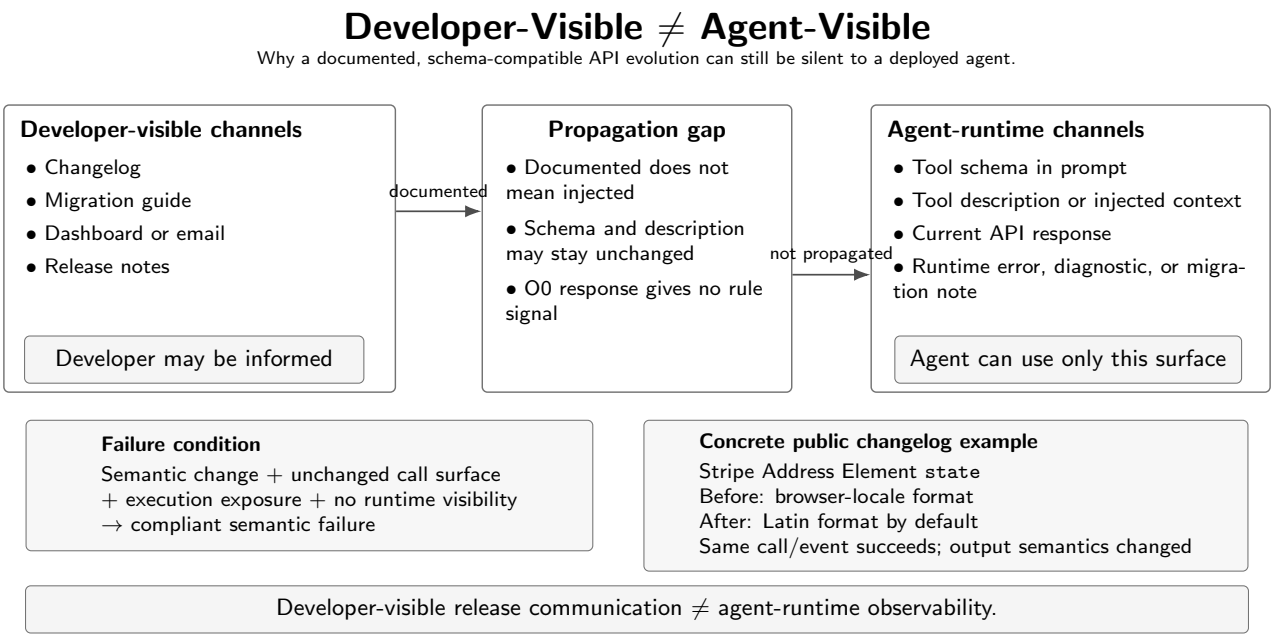

Fig. 1. Developer-visible release communication is not the same as agent-runtime observability. The inset shows a public Stripe changelog example, also replayed in Finding 4, in which the same Address Element state call/event succeeds but the default output semantics change from browser-localized to Latin-formatted output [6]. Compliant semantic failure occurs when such changes are execution-exposed but absent from agent-runtime channels.

face, schema-contract, semantic-contract, and protocol/operational classes, evaluate them over baseline-successful trajectories, and ground the taxonomy in public API-evolution records and real-changelog-derived replay cases.

3) **Empirical evidence on exposure and observability.** Across controlled retail and airline mutations, real-changelog-grounded replays, and non-obviousness controls, we show that execution exposure and semantic observability drive whether schema-compatible semantic changes become recoverable or agent-breaking.

We close by deriving implications for agentic CI/CD: semantic compatibility gates should complement schema checks for business-rule, default, and policy changes that agents may rely on.

II. Agent-Facing Compatibility

A. Why Tool APIs Differ for Agent Clients

Tool-using LLM agents solve tasks by interleaving natural-language dialogue, hidden reasoning, and structured tool calls. They have been studied through reasoning-and-acting prompting, self-supervised tool use, API-call generation, and large-scale tool-use benchmarks [7]–[11]. The tool schema constrains what the model can call, but it does not fully determine how the model interprets the tool. Descriptions, examples, error messages, task wording, retrieved context, and previous execution feedback all influence whether the model chooses the right tool with the right arguments at the right time.

We build on TAU-BENCH retail and airline [11], two task-oriented environments in which an agent assists a simulated user by calling domain tools. Retail tasks involve order lookup, returns, exchanges, payments, and product details. Airline tasks involve reservations, passenger updates, flight changes, baggage changes, and booking policies. These domains are useful because many relevant compatibility obligations are business-semantic rather than purely syntactic.

B. Compatibility Layers

Traditional API compatibility is layered. *Schema-level compatibility* asks whether the machine-readable request and response contracts remain backward compatible for existing valid calls. *Typed-client compatibility* asks whether generated or hand-written clients continue to compile and issue their existing calls. *Behavioral compatibility* asks whether existing calls preserve their documented effects. These layers remain valuable, but they were designed for clients whose behavior is explicit in code and tests.

Agent-facing compatibility adds a task-level layer for agent clients. For an agent, task, baseline API, and evolved API, an evolution is agent-facing compatible when the agent succeeds under the evolved API whenever it succeeded under the baseline API in a paired evaluation. An agent-facing regression occurs when a baseline-successful agent-task-seed cell fails under the evolved API. This definition intentionally conditions on baseline success: compatibility is measured only on tasks the agent could solve before the API evolution.

The key operational concept is *execution exposure*. A changed tool, field, response, or semantic rule is exposed when the baseline trajectory calls the tool, observes the affected

state, or relies on the changed rule to complete the task. A mutation to an unused tool can be schema-breaking in isolation yet irrelevant to a particular task, while a schema-invisible rule change can be task-breaking when it lies on the execution path.

C. Running Example: Developer-Visible Is Not Agent-Visible

Fig. 1 separates developer-visible release communication from agent-runtime observability. A changelog can make a change developer-visible while the deployed agent remains unaware unless the change is propagated into the tool description, injected context, or current API response. Our experiments therefore distinguish O0 silent drift from O1–O4 visible conditions rather than assuming all documented changes are visible to the agent. A *visible policy feedback* case occurs when the evolved API exposes the changed rule through a runtime error, diagnostic, migration note, or updated tool description. The first call may fail, but the agent has a recovery channel: it can revise the plan, ask for missing information, use a different tool path, or escalate.

A *silent semantic drift* case occurs when the same change is documented outside the agent's runtime context or enforced only through final-state semantics. The endpoint and schema remain callable, the local response does not identify the changed rule, and the agent continues a baseline-valid workflow. The first observable failure may be the final-state oracle, a downstream business invariant, or a delayed enforcement stage rather than the API call itself. For example, Stripe's 2026 Address Element changelog changes the default `state` output from browser-localized to Latin-formatted while the same `getValue()` call/event continues to return a string [6]. The call need not fail; the semantic/default-formatting rule changes unless propagated into the agent's runtime context. We use this public changelog example as an additional deterministic replay case in Finding 4. This paper focuses on that compatibility gap: execution-exposed semantic changes that remain absent from the agent's runtime observation.

III. Study Design

A. Mutation Taxonomy and Observability Levels

We study four classes of API evolution: surface/representation changes, schema-contract changes, semantic-contract changes, and protocol/operational changes. Our mutation taxonomy uses mutation testing as a controlled way to study failure modes while targeting API-evolution patterns observed in software ecosystems [12]–[14]. Table I summarizes the taxonomy. We use the A/B/C/D taxonomy as an organizing frame, but the empirical focus is deliberately on C-class semantic-contract changes. A and B changes are often schema-visible or detectable through conventional contract checks, D changes usually require protocol-specific instrumentation, and C1–C4 form the schema-invisible region where endpoint, schema, and call signature can remain stable while task semantics change. This high-risk blind spot is the region evaluated in our main experiments.

For C4 business-rule drift, we evaluate five semantic observability levels while holding the underlying policy change fixed. O0 silent exposes no visible runtime signal and relies on a hidden oracle for the evolved rule. O1 generic error exposes a non-actionable failure. O2 policy error names a policy violation. O3 structured policy error provides diagnostics such as the changed rule, offending field or action, and a recovery hint. O4 migration note exposes a policy diff or migration note before or during task execution. The legacy C4a visible policy violation condition maps to O2-like visible policy error, while legacy C4b silent business-rule drift maps to O0 silent drift.

B. Public API-Evolution Grounding

We ground the taxonomy in a public API-evolution corpus of 151 official changelog, release-note, migration-guide, and API-documentation entries from nine providers: Anthropic, GitHub, Google Cloud, OpenAI, Shopify, Slack, Square, Stripe, and Twilio. We classify each entry into the A/B/C/D taxonomy and identify 61 C-class schema-invisible semantic-contract candidates spanning unit/scale, currency/locale, default-behavior, and business-rule or policy changes. We select grounded replay cases only when the public record provides clear before/after semantics and a deterministic local oracle can be defined. This corpus is not intended to estimate production incident rates; it grounds the mutation taxonomy and case selection in public API-evolution records.

C. Exposure-Aware Paired Protocol

Our protocol is paired and exposure-aware. It follows the spirit of regression testing: evaluate whether behavior that previously succeeded remains valid after a change [5], [15]. Fig. 2 summarizes the paired protocol: retain baseline-successful trajectories, map execution exposure, inject controlled semantic changes on exposed paths or unused-tool controls, vary semantic observability, and rerun the same task/model/seed cell.

First, we run the baseline API and retain only model-task-seed cells in which the agent succeeds. Second, we extract the baseline trajectory to identify which tools and semantic rules are execution-exposed. Third, we rerun the same model, task, and seed under the evolved API and compare the final reward and semantic-oracle flags. Unused-tool negative controls check whether a wrapper or mutation globally destabilizes the environment rather than affecting only exposed paths. For the exposure-control result, each frozen O0 silent cell is matched with an unused-tool control that preserves the domain, model, task, and seed style while placing the same C4 business-rule drift on a tool or rule that the baseline trajectory did not execute or rely on.

This protocol avoids treating all API changes as equally risky. A mutation can be harmless for a task when the changed tool is never called, and harmful when the same changed rule is on the successful baseline path. Agent-facing compatibility is therefore measured over baseline-successful paired cells rather than over standalone schema diffs or single-call validity.

TABLE I
MUTATION TAXONOMY FOR EVOLVING TOOL APIS. C1–C4 FORM THE MAIN HIGH-RISK SCHEMA-INVISIBLE SEMANTIC-CONTRACT REGION.

Class	Mutation examples	Primary API surface	Traditional status	Agent-facing risk hypothesis
A. Surface / representation	`A1_identifier_rename`; `A2_format_change`; `A3_paraphrase`	Names, formatting, descriptions	P/Y depending on mutation	May confuse natural-language tool selection even when meaning is preserved.
B. Schema-contract	`B1_type_change`; `B2_requiredness_change`; `B3_enum_change`; `B4_output_schema_change`	JSON schema, required fields, enums, response shape	Often N or P	Static checks often detect these, but agents may recover through visible validation errors.
C. Semantic-contract	`C1_unit_scale_drift`; `C2_currency_locale_drift`; `C3_default_behavior_drift`; `C4_business_rule_drift`	Hidden semantics and domain rules	Y for selected schema-invisible changes	Schema-level compatible and potentially silent; high risk when execution-exposed.
D. Protocol / operational	`D1_error_format_change`; `D2_permission_change`; `D3_pagination_change`; `D4_rate_limit_change`	Errors, permissions, pagination, rate limits	Y/P/N depending on mutation	Risk depends on whether runtime feedback is visible and actionable.

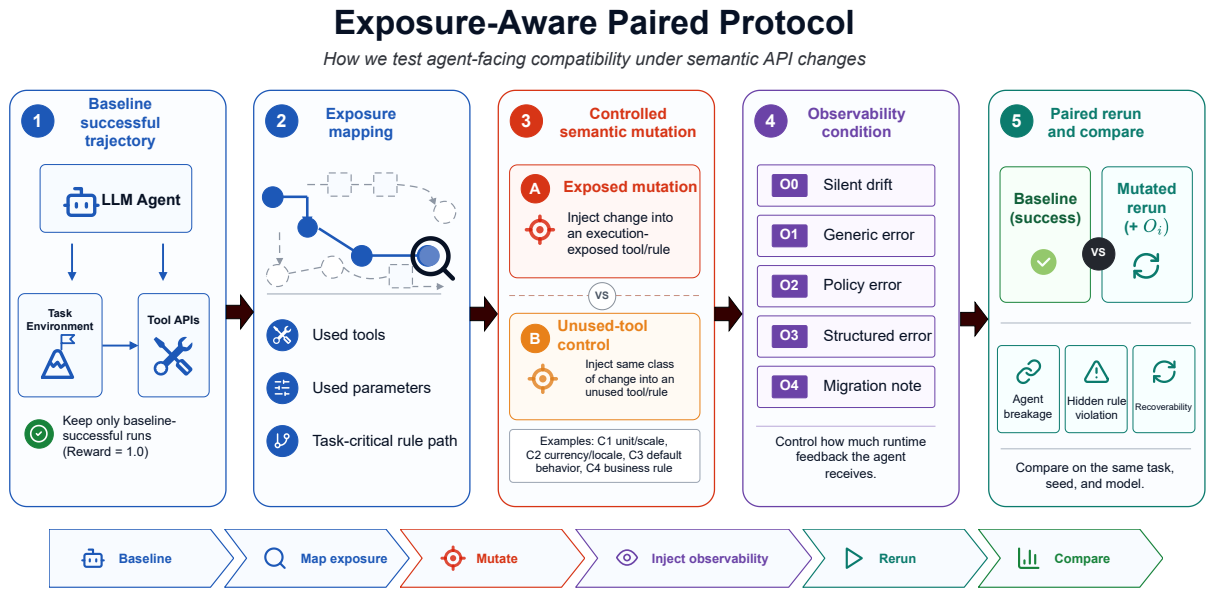

Fig. 2. Exposure-aware paired protocol. Baseline-successful trajectories define execution exposure; controlled semantic mutations and O0–O4 observability levels are replayed on the same task, model, and seed.

D. Environments, Models, and Tasks

The experiments use TAU-BENCH retail and airline (`tau_bench` v0.1.0), with reproduction metadata recorded in the artifact manifest. The retail observability gradient uses candidate tasks 0–19 with seeds 0, 1, and 2; after baseline-successful filtering it contains 105 baseline cells and 525 formal O0–O4 paired cells. The airline gradient uses candidate tasks 0–49 with the same seeds; after filtering it contains 258 baseline cells and 1,290 formal O0–O4 paired cells.

The main formal observability study uses four agent models: `deepseek/deepseek-v4-flash`, `dashscope/qwen-max`, `dashscope/kimi-k2.6`, and `dashscope/glm-5.1`. These identifiers are the exact provider model strings used in the recorded run metadata. The simulated user is `dashscope/qwen-flash`. Run metadata record the provider model string, decoding configuration when available, timeout, and step budget for each cell; the formal paired runs use `max_num_steps=30` and a per-cell timeout of 600 seconds. We report provider model strings and keep supplemental provider extensions outside the frozen main analysis. A supplemental GPT-family airline extension retained in the artifact follows the same silent-versus-visible pattern but is not pooled into the frozen main analysis.

E. Metrics and Data Integrity

We report task success, agent-facing breakage, visible policy feedback, migration-note visibility, hidden business-rule violation, and O4–O0 uplift with bootstrap confidence intervals. Predictor and gate analyses are treated only as artifact-level

engineering evidence; they are not part of the frozen main result.

Different findings use different filtered subsets of the same mutation framework. The schema/client-compatible region in Finding 1 is a compatibility-label slice over 130 strict paired records used to test whether traditional compatibility labels predict agent breakage. The observability gradient in Finding 3 starts from baseline-successful retail and airline cells and expands each cell across O0–O4, yielding 525 retail and 1,290 airline formal paired cells. The C1–C3 semantic-class follow-up uses the same frozen formal models and baseline-successful filtering, adding 480 formal cells that compare O0 silent drift with O3 structured feedback for unit/scale, currency/locale, and default-behavior semantic changes. We keep these denominators separate because they answer different questions: compatibility-label coverage versus recovery under controlled observability.

Formal summaries exclude smoke rows, fake rows, provider-error rows, timeout rows, failed infrastructure rows, and baseline-failed cells. Retry rows are deduplicated by `cell_key`; infrastructure statuses are not counted as agent-facing regressions. Because paired cells share tasks and models across conditions, the artifact additionally reports cluster-bootstrap sensitivity analyses over task or reference-cell clusters; the headline effects remain directionally unchanged. We also generate an oracle audit packet covering 180 baseline, O0-positive, O0-negative, and recovered samples; the automatic audit finds no baseline oracle violations and no automatically suspicious samples. We use this as a deterministic sanity check and release the packet for review; it is not a human-labeled agreement study. The artifact manifest records provenance and inclusion/exclusion rules.

F. Artifact and Reproducibility

To make the paired protocol auditable, the artifact preserves the filtered cell manifests, review packets, inclusion/exclusion rules, mutation specifications, and figure/table generation scripts used for the reported results. The anonymized artifact is available at https://anonymous.4open.science/r/agent-facing-compatibility-artifact-A3A1.
The exposure-control and C1–C3 generalization studies each produce review packets that record planned cells, completed cells, infrastructure statuses, deduplication rules, and aggregate summaries. We also release AFC-Gate as a lightweight implementation of the exposure-aware compatibility workflow: it consumes baseline trajectories and API-change specifications, computes execution exposure, identifies potential compliant semantic failures, and emits targeted replay plans and compatibility reports. The toolkit is intended as an artifact implementation of the workflow rather than a separately evaluated production system.

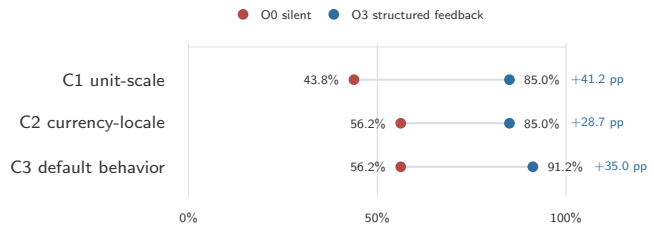

Fig. 3. Visible feedback substantially improves recovery for three schema-invisible semantic-contract changes beyond C4: unit/scale, currency/locale, and default behavior.

IV. Results

A. Finding 1: Compliant Semantic Failures Occur Under Schema-Compatible Evolution

The controlled mutation stress-test identifies a region where traditional schema/client compatibility would not predict a task failure. In the strict schema/client-compatible region, 44 of 130 paired records are agent-breaking (33.8%). Under a relaxed Y+P compatibility label, 49 of 156 paired records are agent-breaking (31.4%). Excluding the A2 format-change relabeling leaves the strict result unchanged at 44 of 130. These are compliant semantic failures: every schema/client-facing surface remains compatible, but the evolved semantic rule invalidates the task outcome. This result identifies a compatibility region that remains invisible to schema/client checks but visible at the agent task level.

Static schema-level and typed-client checks remain useful for schema-visible changes, but they do not detect schema-invisible semantic drift. Our checker compares function names, parameter names, required fields, parameter types, enum values, and array item types; it intentionally ignores natural-language descriptions and vendor extension fields. Consequently, C1–C4 semantic mutations can pass this checker when they preserve the machine-readable schema and call signature, even when the paired agent task fails.

A follow-up C-class generalization study shows that this pattern is not limited to C4 business-rule drift. Across 480 additional formal cells covering C1 unit/scale drift, C2 currency/locale drift, and C3 default-behavior drift, O0 silent success remains low while schema and call signatures stay unchanged. Under O0, success is 43.8% for C1, 56.2% for C2, and 56.2% for C3, with hidden-violation rates of 50.0%, 40.0%, and 36.2%, respectively. With visible feedback, success rises to 85.0%, 85.0%, and 91.2% for C1–C3.

B. Finding 2: Execution Exposure Determines Whether Semantic Drift Becomes Harmful

To test whether silent semantic drift fails because it is execution-exposed rather than because the mutation wrapper globally destabilizes the environment, we construct a matched unused-tool control for every frozen O0 silent cell. Each control keeps the same domain, model, task, and seed style, but places the silent C4 business-rule drift on a tool or rule that the baseline trajectory did not execute or rely on.

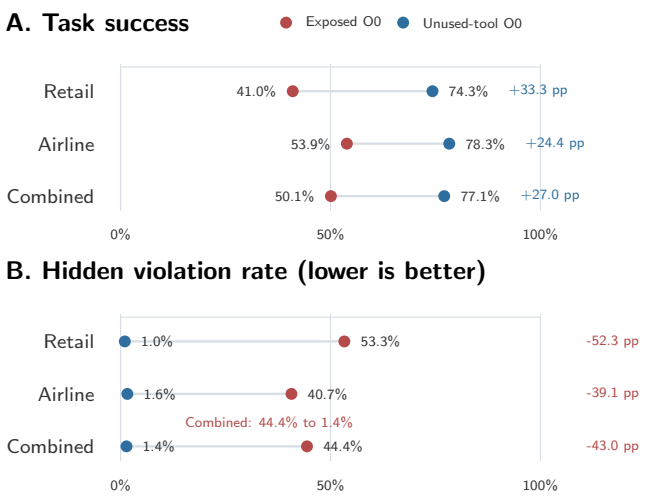

Fig. 4. Execution exposure determines whether silent semantic drift becomes harmful. Matched unused-tool controls raise task success from 50.1% to 77.1% and reduce hidden violations from 44.4% to 1.4%, showing that the same class of silent drift is far less harmful when placed off the agent's execution path.

Across 363 exposed O0 cells and 363 matched unused-tool controls, the unused-tool condition succeeds substantially more often: 77.1% versus 50.1% for exposed O0. Hidden business-rule violations drop from 44.4% under exposed O0 to 1.4% under unused-tool control. The success-rate difference is +27.0 percentage points with bootstrap 95% CI [+20.1, +33.6]; a task-cluster bootstrap sensitivity check preserves the direction with CI [0.150, 0.378]. Fisher's exact test gives $p = 4.38 \times 10^{-14}$ as a descriptive row-level check. The target unused tool is reached in only 5 of 363 controls (1.4%).

This result strengthens execution exposure as the mechanism behind agent-facing compatibility risk. The same silent semantic drift is much less harmful when it is placed off the agent's execution path, ruling out a wrapper-global destabilization explanation. The direction is consistent in both domains: retail rises from 41.0% exposed success to 74.3% unused-control success, and airline rises from 53.9% to 78.3%. All four frozen formal models show the same direction: unused-tool controls have higher success and far lower hidden-violation rates than exposed O0.

C. Finding 3: Semantic Observability Creates a Recovery Channel

Using the five observability levels defined in Section III, we evaluate 525 formal retail cells and 1,290 formal airline cells, for 1,815 frozen main paired cells. All formal cells pass the integrity rules in Section III. Fig. 5 summarizes the O0–O4 gradient and the O4–O0 uplift in one view.

Across 1,815 frozen paired cells, combined success rates for O0–O4 are 0.501, 0.843, 0.846, 0.851, and 0.848. Retail rises from 0.410 under O0 to 0.857 under O4; airline rises from 0.539 to 0.845. The O4–O0 uplift is 0.448 in retail with bootstrap 95% CI [0.333, 0.562], 0.306 in airline with CI [0.233, 0.380], and 0.347 combined with CI [0.289, 0.405]. The dominant empirical effect is the transition from O0 silent drift to any visible condition. Once the evolved rule becomes visible, even generic feedback can provide a recovery channel; richer structured diagnostics and migration notes remain useful compatibility metadata, but their marginal gains over basic visibility are smaller and vary by model and domain. The engineering message is therefore not that every richer feedback format strictly dominates simpler feedback, but that hiding semantic change behind an unchanged schema removes the agent's recovery channel. In the plateau analysis, O0 success is 50.1% versus 84.7% for any visible O1–O4 condition, an uplift of +34.6 percentage points with bootstrap 95% CI [+29.7, +39.5]; a task-cluster bootstrap sensitivity check remains positive with CI [0.216, 0.493]. By contrast, O1–O4 marginal differences are all within about one percentage point in the aggregate and are not strictly monotonic.

The same silent-versus-visible pattern generalizes beyond C4, as shown in Fig. 3. This supports semantic observability as a general recovery mechanism for schema-invisible semantic-contract changes.

Trace example: silent failure and visible recovery.

Condition	Agent action	API feedback	Outcome
O0 silent	Valid booking call mixes certificate $250 with card $5.	No visible policy error; hidden oracle checks evolved payment rule.	Reward 0; hidden violation.
O3 structured	After feedback, agent retries with card-only payment.	Structured payment error suggests a single payment method.	Reward 1; no hidden violation.

This controlled TAU-BENCH trace illustrates the same observability mechanism as the grounded replays. A representative trace illustrates the aggregate pattern: under O0, the agent issues a valid call and receives no recovery signal; under visible feedback, the agent can revise the workflow.

Trace-level interpretation. The trace cases clarify why the aggregate O0–visible gap is not merely a scoring artifact. In the O0 case, the agent issues a syntactically valid booking call that matches the unchanged schema and receives no policy-level correction, so the local interaction appears successful even though the final state violates the evolved payment rule. The failure is therefore compliant at the call surface but semantic at the task level. In the O3 case, the API surfaces the changed payment constraint as structured feedback; the agent can revise the workflow and retry with a valid payment strategy. This illustrates the mechanism behind the plateau result: once the evolved rule becomes observable, the agent receives a recovery channel that silent drift removes.

The earlier binary C4a/C4b retail and airline results show the same direction and are retained in the artifact as secondary traceability evidence. Per-model rates are also reported in the artifact rather than duplicated in the main paper.

D. Finding 4: Grounded Replays Show Observability, Not Extra Reasoning, Drives Recovery

The public API-evolution corpus contains 151 entries across nine official providers, including 61 C-class schema-invisible

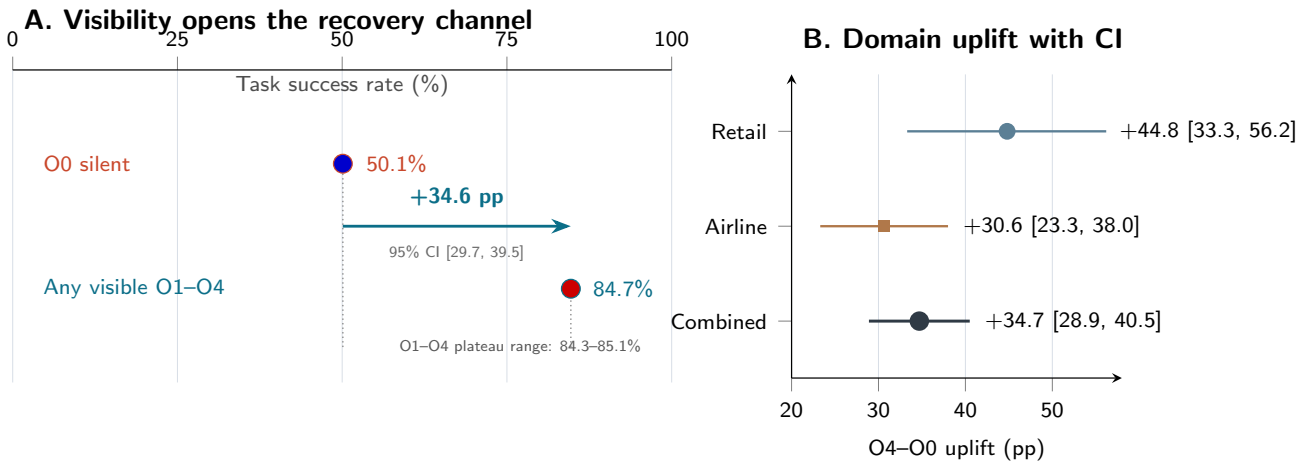

Fig. 5. Semantic observability restores the agent's recovery channel. Panel A highlights the dominant transition from O0 silent drift to the visible-feedback plateau: combined success rises from 50.1% under O0 to 84.7% under any visible condition. Panel B shows O4–O0 uplift with bootstrap 95% confidence intervals across domains.

semantic-contract candidates. From the public corpus, we replay three public-changelog-grounded semantic changes with clear before/after semantics and deterministic local oracles: a Stripe-derived C1 validation-semantics change, a Stripe-derived C4 payment restriction, and a Stripe Address Element default-formatting case in which the same call/event succeeds while output semantics change. These are deterministic local wrapper replays, preserving the schema-visible call surface while varying whether the evolved semantic rule is silent or visible. The validation case is replay evidence under this wrapper design, not the Fig. 1 silent headline. The source changelog entries and extraction notes are included in the anonymized artifact. The replay is not a live production-service experiment. Its purpose is to ground the semantic change, not to measure Stripe incident rates: the before/after rule is taken from public API-evolution records, while the deterministic wrapper isolates the agent-facing compatibility mechanism under controlled execution.

TABLE II
GROUNDING AND NON-OBVIOUSNESS CONTROLS. REAL-CHANGELOG REPLAYS USE PUBLIC API-EVOLUTION RECORDS AND DETERMINISTIC LOCAL WRAPPERS; CONTROLS TEST WHETHER EXTRA REASONING RECOVERS WITHOUT VISIBILITY.

Study	Condition	Success	Hidden violation
Replay	Baseline old API	36/36	0/36
Replay	Evolved O0 silent	6/35	29/35
Replay	Evolved visible	34/35	0/35
Control	O0 + more reasoning	3/96	92/96
Control	O0 + reflection	6/95	88/95
Control	Rule visible upper bound	71/95	0/95

Note: The 288-cell control has 286 OK rows, and the real replay has 106 OK rows from 108 terminal rows. Non-ok infrastructure rows are excluded and not counted as agent failures.

The real-changelog-grounded replays reproduce the same mechanism as the controlled mutation study. Under old semantics, all 36 baseline cells succeed with no hidden violations. Under evolved O0 silent semantics, success falls to 6 of 35 and hidden violations rise to 29 of 35. Under evolved visible feedback, 34 of 35 cells succeed, no hidden violation fires, and the changed rule is exposed in all 35 cells. This upgrades the claim from constructing such failures to showing that public API-evolution records contain the same semantic-change pattern and that grounded before/after replays reproduce the mechanism.

The non-obviousness control rules out the simple explanation that agents merely need more reasoning budget or a reflection scaffold. With O0 silent drift, increasing the reasoning budget succeeds in only 3 of 96 cells and leaves hidden violations in 92 of 96. A reflection scaffold succeeds in 6 of 95 and leaves hidden violations in 88 of 95. When the same evolved rule is made visible as an upper bound, success rises to 71 of 95 and hidden violations fall to 0. Reasoning helps agents use visible signals, but it cannot reliably infer an unobserved external business-rule change from an unchanged call surface.

V. Discussion and Implications

A. Observability as Compatibility Metadata

The central mechanism in our results is semantic observability. The relevant question is not whether a human developer could eventually read a changelog; it is whether the deployed agent receives the changed rule through the runtime channel it actually uses. When an evolved rule becomes visible through an error, structured diagnostic, or migration note, the agent gains a recovery channel that silent drift removes. A generic error can already signal that a baseline-valid workflow no longer works, while structured diagnostics and migration notes make the changed policy easier to incorporate before or after a tool call. For API teams, this suggests that eligibility rules, payment policies, unit changes, default changes, and other semantic evolutions should be surfaced as visible or machine-readable compatibility metadata rather than hidden behind

unchanged schemas, aligning schema-first tool interfaces with API-description standards [1], [16]. The non-obvious result is not that an agent fails when a hidden rule changes. The result is that the dominant improvement comes from making the semantic change observable at all. Extra reasoning and reflection under O0 remain near the silent-failure regime, while even low-cost visibility gives the agent a recovery channel. This suggests that API teams do not always need elaborate agent-specific protocols to avoid the worst failures; they need to avoid silent semantic drift on execution-exposed paths.

B. Exposure-Aware Regression Testing

Agent-facing risk depends on execution exposure. A mutation to an unused tool or unused rule may not affect a given task, whereas the same semantic change can be catastrophic when it lies on a baseline-successful path. Agent platform teams should therefore preserve baseline-successful trajectories and replay them against candidate API evolutions, prioritizing tools and rules that are actually exercised by tasks. This does not replace broad API regression testing; it adds a workflow-level semantic test for the parts of an API that agents rely on.

C. Toward Exposure-Aware Compatibility Gates

The empirical results point to two actionable signals for agent-platform engineering: execution exposure and semantic observability. AFC-Gate operationalizes the empirical workflow rather than serving as a separate evaluated system contribution: given baseline trajectories and API-change specifications, it computes execution exposure, flags potential compliant semantic failures, plans targeted replay, and generates a compatibility report. Its role is to show that agent-facing compatibility can be turned into an automated testing workflow; the core empirical claims come from the paired studies and real-changelog-grounded replays above. The design follows directly from the empirical findings: schema-only checks are insufficient for schema-invisible semantic drift, while exposure-aware replay targets the tasks and rules that agents actually exercise.

D. Engineering Guidelines

The results suggest several engineering practices for agent-facing API evolution. First, treat schema-invisible semantic changes as compatibility-relevant whenever they affect rules, defaults, units, currencies, eligibility, or payment behavior that agents may rely on. Second, preserve baseline-successful agent trajectories and use them as regression assets; compatibility should be checked on workflows agents actually complete, not only on isolated tool calls. Third, map candidate API changes to execution exposure before deciding replay scope, so that off-path changes do not dominate testing cost while on-path semantic changes receive priority. Fourth, avoid O0 silent drift for execution-exposed semantic changes: at minimum, provide visible policy feedback, structured diagnostics, or migration notes that agents can use to revise their plans. These guidelines do not replace schema checks; they extend them with task-level semantic evidence for agent clients.

VI. Threats to Validity

Controlled environments. The study uses controlled TAU-BENCH retail and airline environments rather than production logs. Real API evolution, versioning, and breaking-change practices are known to be complex in software ecosystems and microservice settings [14], [17], [18]. Rates such as 44/130 and the observed O4–O0 uplift should be read as evidence of a compatibility mechanism under controlled API evolution, not as production base rates. The airline gradient reduces the risk that the main mechanism is retail-specific, but both domains are still benchmark environments.

Mutation realism and C4b construction. The O0 silent condition is designed to isolate the hardest case: a schema-compatible semantic rule changes without a visible recovery signal. Therefore, the paper does not treat low O0 recovery alone as surprising. The empirical content lies in the silent-versus-visible gap, the unused-tool negative control, cross-domain replication, and the observation that generic visibility already recovers much of the lost task success. The exposure-control experiment strengthens the mechanism claim, but production API ecosystems may expose additional hidden dependencies that require richer exposure extraction. The public-changelog corpus and grounded replays improve realism but do not estimate production incident frequency. The replays use deterministic local wrappers derived from public before/after semantics rather than live third-party services, which lets us isolate agent-facing compatibility while avoiding external service nondeterminism. We replay only cases with clear deterministic before/after semantics; candidates such as a GitHub C3 default-behavior change remain artifact-review cases when local evidence is insufficient for faithful replay.

Model, provider, and user simulator limits. The frozen main observability study uses four available formal agent models and a DashScope `qwen-flash` user simulator. These choices may not represent all commercial, open, or production agent systems. Provider-side model snapshots may drift over time; the artifact records model strings and run metadata where available. A supplemental GPT-family airline extension retained in the artifact follows the same silent-versus-visible pattern, but it remains outside the frozen main analysis.

Protocol and labeling limits. Baseline-successful filtering is necessary for a paired compatibility definition, but it shifts attention to tasks agents can already solve under the baseline API. The static checker is a deliberate approximation of schema-level and typed-client compatibility; richer industrial checkers may catch more changes but still need semantic evidence for schema-invisible rule drift. The study covers C1–C4 semantic-contract changes but remains centered on schema-invisible semantic drift rather than a full A/B/D taxonomy sweep. The oracle audit packet contains 180 automatically checked samples with no baseline oracle violations and no automatically suspicious samples, but it is a human-review-ready audit rather than a human-labeled agreement study. AFC-Gate remains an artifact implementation of the workflow rather than a complete industrial certification mechanism;

production use would require richer policy oracles, monitoring, and organization-specific rollout controls. These choices bound the scope of the claim; they do not undermine the mechanism because the controlled study, exposure negative control, public corpus, grounded replay, and non-obviousness control all point to the same silent-versus-visible mechanism.

VII. Related Work

A. API Compatibility and Regression Testing

API compatibility research studies how interfaces evolve and how clients experience breaking changes. Interface-description and schema standards such as OpenAPI and JSON Schema make request/response contracts machine-readable, while semantic versioning provides a convention for communicating compatibility expectations [1]–[3]. Empirical studies and surveys show that API evolution remains difficult in practice, including breaking changes, versioning practices, microservice coordination, and ecosystem-level impact [4], [5], [14], [17], [18]. Recent dependency-upgrade work also studies compatibility-preserving mitigation of technical lag: DepUpdater targets Java projects by reducing technical lag while avoiding newly introduced breaking changes and redundant dependencies [19]. This line of work focuses on code-level dependency management for conventional projects; our setting shifts compatibility to interactive tool-using agents, where the call surface can remain valid while task-level semantics evolve silently.

Regression testing and mutation testing provide the testing foundation for our paired protocol. Regression testing asks whether previously valid behavior remains valid after a change, while mutation testing uses controlled perturbations to expose test-suite and system weaknesses [12], [13], [15]. Our work applies these ideas to tool APIs consumed by LLM agents, where the compatibility surface includes schema, semantics, observability, and execution exposure.

Recent work studies LLMs under API or library evolution in code generation, including deprecated API usage in LLM-based completion and conflicts between updated API context and stale parametric knowledge [20]–[22]. We instead study interactive tool-using agents whose calls remain syntactically valid while task success changes because an execution-exposed business rule evolves silently.

B. Tool-Using Agent Evaluation

Tool-using LLM agents have been studied through reasoning-and-acting prompting, self-supervised tool-use learning, API-call generation, function-calling benchmarks, and large-scale agent environments [7]–[10], [23], [24]. Recent benchmarks add stateful tool interaction, simulated users, app ecosystems, enterprise workflows, and MCP-server tasks, including TAU-BENCH, ToolSandbox, AppWorld, WorkArena, and MCP-Bench [11], [25]–[28]. These benchmarks primarily ask whether an agent can solve tasks under a fixed API; we ask whether API evolution preserves success for baseline-successful agent-task-seed cells.

Reasoning and prompting strategies can improve how agents plan, reflect, and use visible feedback, but they do not remove the need for API-side observability. In the O0 setting studied here, the call is syntactically valid, the schema is unchanged, and the agent receives no policy error; a stronger planner may use visible diagnostics more effectively, but it cannot reliably infer an unobserved external rule change from the unchanged call surface alone.

C. Tool Misuse, Guardrails, and Observability

Recent work on schema-first tool APIs and tool-misuse benchmarks studies how interface design affects tool misuse and recovery [16], [29]. Agent safety, dynamic replanning, and recovery benchmarks study unsafe actions, explicit tool failures, anomaly recovery, and stress conditions [30], [31]. Runtime guardrail and provenance work studies how to bound or monitor agents during execution [32]. Our focus is complementary: the agent may issue a syntactically valid call that was appropriate under the old semantics, while the API evolves underneath it. Observability matters because visible errors, structured diagnostics, migration notes, and semantic oracles can convert silent drift into recoverable compatibility metadata.

Unlike tool-misuse benchmarks, our failure mode does not require an invalid tool call: the call is valid under an unchanged schema, and the regression arises because the API's business semantics evolve. Unlike state-diff benchmarks under fixed APIs, we study whether API evolution preserves success for baseline-successful agent trajectories. Tool-misuse and guardrail work often focuses on invalid calls, unsafe actions, out-of-bounds behavior, or recovery from explicit tool failures; our central failure mode is semantic and compatibility-driven.

VIII. Conclusion

Tool-using LLM agents make API compatibility a task-level semantic problem. Our results demonstrate that relying only on schema stability leaves agentic systems vulnerable to compliant semantic failures: syntactically valid calls to unchanged schemas can silently violate evolved business rules. The C1–C3 follow-up shows that this risk extends beyond C4 business-rule drift to broader schema-invisible semantic-contract changes. Public API-evolution grounding and three real-changelog-derived replays show that the mechanism is not limited to hand-written mutations, while the non-obviousness controls show that additional reasoning without semantic visibility does not close the gap. Across controlled retail and airline experiments, execution exposure determines whether such changes matter, and semantic observability determines whether agents receive a recovery channel. The matched unused-tool controls show that this risk is not a global artifact of mutation wrappers: semantic drift becomes harmful primarily when it is execution-exposed. The empirical signal is substantial: even within the strict schema/client-compatible region, 44 of 130 paired records are agent-breaking (33.8%), exposing a class of failures that conventional compatibility tooling does not target.

Agent-facing compatibility therefore calls for exposure-aware semantic testing in addition to schema checks. The engineering implication is direct: CI/CD pipelines for agentic systems need semantic compatibility gates for business-rule, default, and policy changes that agents may rely on. Together, the empirical results and artifact implementation show that agent-facing compatibility can be operationalized as exposure-aware semantic regression testing. Our proof-of-concept gating analysis shows one path toward such pipelines, while the core empirical result is broader: compatibility depends on what the agent executes and what the evolved API makes observable. Future work should broaden the domains, add more protocol and schema-contract diversity, and validate these mechanisms against production evolution logs where available.

REFERENCES

[1] OpenAPI Initiative, "OpenAPI specification, version 3.1.0." https://spec.openapis.org/oas/v3.1.0.html, 2021. Accessed 2026-06-18.

[2] JSON Schema, "JSON Schema Draft 2020-12." https://json-schema.org/draft/2020-12/json-schema-core.html, 2022. Accessed 2026-06-18.

[3] T. Preston-Werner, "Semantic Versioning 2.0.0." https://semver.org/spec/v2.0.0.html, 2013. Accessed 2026-06-18.

[4] M. Lamothe, Y.-G. Guéhéneuc, and W. Shang, "A systematic review of API evolution literature," *ACM Computing Surveys*, vol. 54, no. 8, 2021.

[5] L. Xavier, A. Brito, A. Hora, and M. T. Valente, "Historical and impact analysis of API breaking changes: A large-scale study," in *Proceedings of the 24th IEEE International Conference on Software Analysis, Evolution and Reengineering*, SANER, pp. 138–147, 2017.

[6] Stripe, "Changes the address element state field to default to latin-formatted characters." https://docs.stripe.com/changelog/dahlia/2026-03-25/address-element-getvalue-and-change-event-formatting, 2026. Developer changelog, accessed 2026-06-24.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[9] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," 2024.

[10] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *International Conference on Learning Representations*, 2024.

[11] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," 2024.

[12] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[13] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: An analysis and survey," in *Advances in Computers*, vol. 112, pp. 275–378, Elsevier, 2019.

[14] A. Lercher, J. Glock, C. Macho, and M. Pinzger, "Microservice API evolution in practice: A study on strategies and challenges," *Journal of Systems and Software*, vol. 215, p. 112085, 2024.

[15] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: A survey," *Software Testing, Verification and Reliability*, vol. 22, no. 2, pp. 67–120, 2012.

[16] A. Sigdel and R. Baral, "Schema first tool APIs for LLM agents: A controlled study of tool misuse, recovery, and budgeted performance," 2026.

[17] S. Serbout and C. Pautasso, "An empirical study of web API versioning practices," in *Web Engineering*, vol. 13893 of *Lecture Notes in Computer Science*, pp. 303–318, Springer, 2023.

[18] L. Ochoa, T. Degueule, J.-R. Falleri, and J. Vinju, "Breaking bad? semantic versioning and impact of breaking changes in maven central: An external and differentiated replication study," *Empirical Software Engineering*, vol. 27, no. 3, p. 61, 2022.

[19] R. Lu, L. Zhang, K. Li, M. Zhang, and Y. Chen, "Minimizing breaking changes and redundancy in mitigating technical lag for java projects," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, ICSE '26, 2026.

[20] C. Wang, K. Huang, J. Zhang, Y. Feng, L. Zhang, Y. Liu, and X. Peng, "LLMs meet library evolution: Evaluating deprecated API usage in LLM-based code completion," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, ICSE '25, 2025.

[21] A. N. Ashik, S. Wang, T.-H. Chen, M. Asaduzzaman, and Y. Tian, "When LLMs lag behind: Knowledge conflicts from evolving APIs in code generation," 2026.

[22] L. Liang, J. Gong, M. Liu, C. Wang, G. Ou, Y. Wang, X. Peng, and Z. Zheng, "RustEvo2: An evolving benchmark for API evolution in LLM-based Rust code generation," 2025.

[23] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *Proceedings of the International Conference on Learning Representations*, 2024.

[24] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, "The Berkeley function calling leaderboard," in *Proceedings of Machine Learning Research*, 2025.

[25] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, F. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang, "ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities," *arXiv preprint arXiv:2408.04682*, 2024.

[26] H. Trivedi, T. Khot, and A. Sabharwal, "AppWorld: A controllable world of apps and people for benchmarking interactive coding agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[27] A. Drouin, M. Gasse, M. Caccia, I. H. Laradji, M. Del Verme, T. Marty, L. Boisvert, M. Thakkar, Q. Cappart, D. Vazquez, N. Chapados, and A. Lacoste, "WorkArena: How capable are web agents at solving common knowledge work tasks?," *arXiv preprint arXiv:2403.07718*, 2024.

[28] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow, "MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers," *arXiv preprint arXiv:2508.20453*, 2025.

[29] A. Sigdel and R. Baral, "ToolMisuseBench: An offline deterministic benchmark for tool misuse and recovery in agentic systems," 2026.

[30] Z. Zhang, S. Cui, Y. Lu, J. Zhou, J. Yang, H. Wang, and M. Huang, "Agent-SafetyBench: Evaluating the safety of LLM agents," *arXiv preprint arXiv:2412.14470*, 2024.

[31] D. Zhu, X. Ma, Y. Shen, X. Li, Y. Zhao, S. Wang, L. Yan, and D. Yin, "When tools fail: Benchmarking dynamic replanning and anomaly recovery in LLM agents," *arXiv preprint arXiv:2606.05806*, 2026.

[32] R. Sequeira, S. Damianakis, U. Iqbal, and K. Psounis, "Agent-Sentry: Bounding LLM agents via execution provenance," 2026.